Task 3

Password Security Audit Tool

1. Introduction

Password security is one of the most important aspects of cybersecurity. Weak passwords can be easily guessed or cracked using brute-force and dictionary attacks, leading to unauthorized access to sensitive information.

The objective of this project is to develop a Password Security Audit Tool using Python that evaluates password strength, detects common passwords, estimates crack time, suggests stronger passwords, and securely hashes passwords using bcrypt.

2. Objectives

The main objectives of this project are:

- Check password strength based on security criteria.
- Verify passwords against a list of 10,000 common passwords.
- Estimate password crack time using brute-force calculations.
- Generate stronger password suggestions.
- Securely hash passwords using bcrypt.
- Improve awareness of password security practices.

3. Tools and Technologies Used

Tool	Purpose
Python	Programming Language
VS Code	Code Editor
bcrypt	Secure Password Hashing
Git & GitHub	Version Control and Repository Hosting

4. Features Implemented

4.1 Password Strength Analysis

The tool checks whether the password contains:

- Uppercase letters
- Lowercase letters
- Numbers
- Special characters
- Minimum recommended length

Based on these criteria, the password is classified as:

- Weak
- Moderate
- Strong

Output Screenshot

```
PS C:\PasswordSecurityTool> python main.py
Enter your password: password123

Password Strength Score: 3 /5
Moderate Password

Estimated crack time: 131621703.84 seconds

Suggested Strong Password:
YDFqgixq!jWI

Secure Hashed Password:
$2b$12$aWw5Dr6WggIBxlyCq9MAf.ALNohOQDqc/c2fbg5RI7y4c0YCbzY0C
PS C:\PasswordSecurityTool> python main.py
Enter your password: MySecure@123Password

Password Strength Score: 5 /5
Strong Password

Estimated crack time: 140168339535626071021222625 28.00 seconds

Suggested Strong Password:
03C69o1KXcSP

Secure Hashed Password:
$2b$12$8qoAhE4a3CiF3nTBoFYQ6eltVbzGGaSTjUzTapT3SrS5vpulo3XOC
PS C:\PasswordSecurityTool>
```

4.2 Common Password Detection

The tool compares user passwords against a database containing the 10,000 most common passwords.

If the entered password exists in the list, a warning message is displayed.

4.3 Crack Time Estimation

The program estimates how long a brute-force attack would take to crack the entered password.

This estimation depends on:

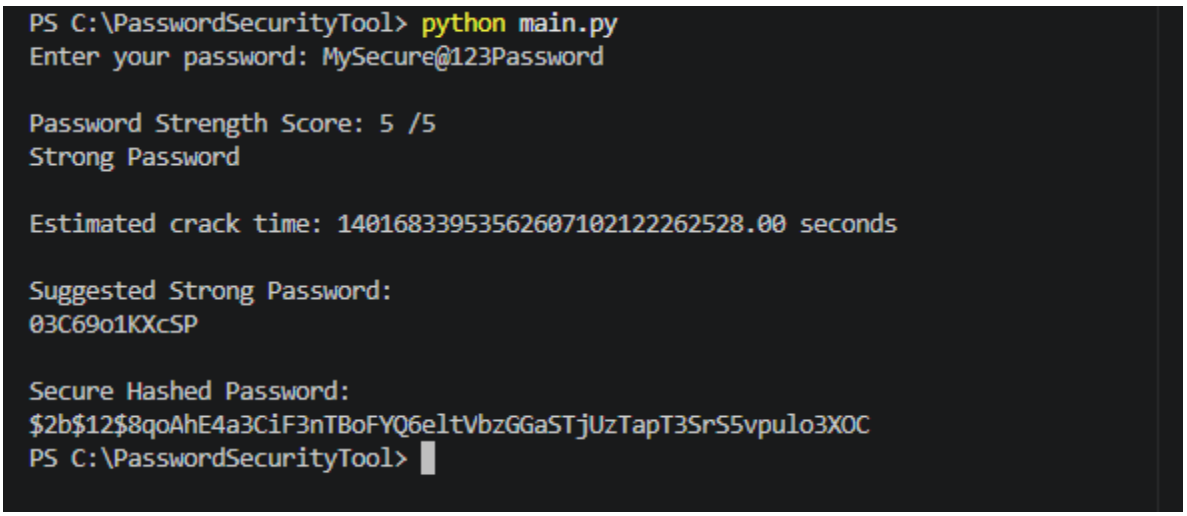
- Password length
- Character variety
- Search space size

4.4 Strong Password Suggestion

When a weak password is detected, the tool generates a stronger alternative containing:

- Uppercase letters
- Lowercase letters
- Numbers
- Special characters

Output Screenshot



4.5 Secure Password Hashing Using bcrypt

Instead of storing passwords in plain text, the tool hashes passwords using bcrypt.

Benefits include:

- Secure storage
- Salt generation
- Protection against rainbow table attacks
- Improved resistance to brute-force attacks

5. Security Findings

During testing, the following observations were made:

- Common passwords are highly vulnerable.
- Longer passwords significantly increase security.
- Combining multiple character types improves strength.
- Password hashing is essential for secure storage.
- bcrypt provides stronger protection than plain-text storage.

6. Challenges Faced

- Installing and configuring Python libraries.
- Understanding bcrypt hashing.
- Implementing password strength calculations.
- Handling common password list integration.

7. Results

The Password Security Audit Tool successfully:

✓ Evaluated password strength

✓ Detected common passwords

✓ Estimated crack time

✓ Suggested stronger passwords

✓ Securely hashed passwords using bcrypt

All project objectives were achieved successfully.

8. Conclusion

This project helped in understanding password security principles and secure password storage techniques. The Password Security Audit Tool demonstrates how weak passwords can be identified and improved while ensuring secure handling of credentials through bcrypt hashing.

The project contributes to cybersecurity awareness and promotes the use of stronger passwords to reduce security risks.

References

1. Python Documentation
2. bcrypt Documentation
3. OWASP Password Security Guidelines
4. Common Password Dataset (10,000 Password List)